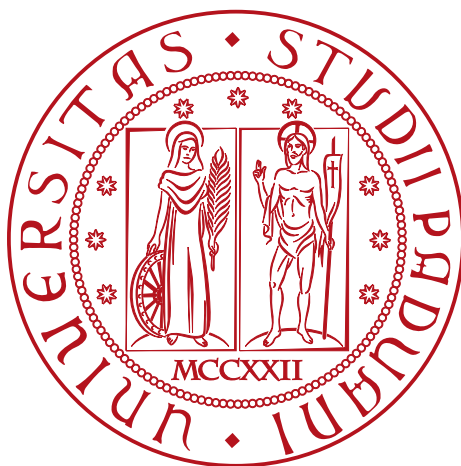


Università degli studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



**Servizio event-driven per l'estrazione
e la sintesi di KPI aziendali tramite
semantic layer e LLM**

Tesi di laurea triennale

Relatore

Prof. Federico Corò

Laureando

Giovanni Ponso

Matricola 2000558

«Frase ad effetto»

— Autore della suddetta frase

Ringraziamenti

Da scrivere al termine della stesura.

Padova, Settembre 2026

Giovanni Ponso

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare, della durata di trecentoventi ore, dal laureando Giovanni Ponso presso l'azienda Datasoil S.r.l.. Lo stage è stato condotto sotto la supervisione del tutor aziendale Pietro De Caro, mentre il Prof. Federico Corò ha ricoperto il ruolo di tutor accademico.

Questa tesi tratta la progettazione e lo sviluppo di un servizio per la generazione automatica di riepiloghi testuali all'interno della piattaforma SaaS di facility management di Datasoil. Il servizio reagisce agli eventi della piattaforma, recupera in modo deterministico i KPI dell'entità interessata tramite un semantic layer e ne affida l'esposizione in linguaggio naturale a un modello linguistico (LLM), con l'obiettivo di supportare l'operatore sul campo con un quadro sintetico e attendibile dello stato di asset, ispezioni e ticket.

Indice

1	Introduzione	1.
1.1	L'azienda	1.
1.2	Il progetto e lo stage	2.
1.2.1	Descrizione	2.
1.2.2	Principali problematiche e relative soluzioni	2.
1.2.3	Pianificazione	4.
1.2.4	Obiettivi	4.
1.3	Il prodotto finale	4.
1.4	Organizzazione del testo	5.
2	Strumenti e tecnologie utilizzate	7.
2.1	Go	7.
2.2	MongoDB	7.
2.3	Cube	8.
2.4	Modelli linguistici e API OpenAI	8.
2.5	Amazon SQS ed ElasticMQ	8.
2.6	Docker	9.
2.7	Librerie di supporto	9.
3	Analisi dei requisiti	11.
3.1	Studio preliminare degli approcci	11.
3.2	Casi d'uso	11.
3.3	Classificazione dei requisiti	11.
3.4	Tracciamento dei requisiti	11.
4	Progettazione e sviluppo	13.
4.1	Architettura del servizio	13.
4.2	Estrazione dei KPI	13.
4.3	Generazione del riepilogo tramite LLM	13.
4.4	Caching e persistenza	13.
4.5	Integrazione con la coda	13.
5	Verifica e validazione	15.

5.1 Approccio alla verifica	15.
5.2 Test del flusso completo	15.
5.3 Validazione con il tutor aziendale	15.
6 Conclusioni	17.
6.1 Raggiungimento degli obiettivi	17.
6.2 Sviluppi futuri	17.
6.3 Conoscenze acquisite	17.
6.4 Valutazione personale	17.
Glossario	19.
Bibliografia	20.

Elenco delle Figure

Elenco delle Tabelle

**Elenco dei Codici
Sorgente**

Capitolo 1

Introduzione

In questo capitolo presento l'azienda ospitante e introduco il progetto di stage: le problematiche affrontate, la pianificazione, gli obiettivi e il prodotto realizzato.

1.1 L'azienda

Datasoil S.r.l. è una software house con sede a Padova, parte del gruppo internazionale eFM. Attraverso la propria piattaforma SaaS B2B, l'azienda porta intelligenza artificiale ed *execution* mobile al centro delle *operations*, collegando mondo fisico e digitale in un unico layer operativo: dalle ispezioni alla manutenzione, fino alla gestione degli asset, le attività frammentate vengono trasformate in processi strutturati, scalabili e governabili. Datasoil opera nei settori *real estate*, industriale e manifatturiero, supportando organizzazioni complesse nell'evoluzione verso modelli operativi più integrati, predittivi e orientati al valore.

Il progetto di stage si colloca nell'ambito del *facility management*, ovvero la gestione integrata di edifici, impianti e asset aziendali, dove la piattaforma ruota attorno a tre entità fondamentali: gli **asset** (i beni gestiti, come impianti e attrezzature), le **ispezioni** (le verifiche periodiche condotte sugli asset) e i **ticket** (le segnalazioni e gli interventi di manutenzione).

Il prodotto è **multi-tenant**: una singola installazione serve più aziende clienti, i cui dati sono mantenuti rigorosamente separati. Come si vedrà nel corso del documento, questa caratteristica ha attraversato l'intero progetto di stage: dall'organizzazione dei dati, alla definizione delle metriche per i singoli clienti, fino alla configurazione delle interrogazioni per ciascun tenant.

1.2 Il progetto e lo stage

1.2.1 Descrizione

Il progetto di stage nasce dall'esigenza, individuata nel piano di lavoro, di efficientare l'operatività dell'utente sul campo: restituire in modo automatico una sintesi dello stato di manutenzione e dei relativi interventi per le diverse tipologie di entità gestite dalla piattaforma.

In concreto, si tratta di sviluppare un servizio che, all'apertura della scheda di un asset (o di un'ispezione o di un ticket), generi automaticamente un **riepilogo testuale in linguaggio naturale** dello stato dell'oggetto, sfruttando i dati aziendali per garantire l'attendibilità del contenuto e un modello linguistico (LLM) per la sua esposizione.

Il servizio sviluppato è di tipo *event-driven*: reagisce agli eventi pubblicati dalla piattaforma su una coda (ad esempio il completamento di un'ispezione), recupera i KPI dell'entità interessata, ne affida la sintesi all'LLM e rende disponibile il riepilogo generato.

Lo stage si è svolto in modalità full-time per un totale di 320 ore, sotto la supervisione del tutor aziendale Pietro De Caro. Gli obiettivi erano fissati dal piano di lavoro concordato con l'azienda, che mi ha lasciato però ampia autonomia sul *come* raggiungerli: per ciascun obiettivo ho valutato approcci alternativi, sviluppato prototipi per verificarne la fattibilità e consolidato le scelte attraverso frequenti momenti di confronto con il tutor.

1.2.2 Principali problematiche e relative soluzioni

Le principali problematiche riscontrate durante la realizzazione del progetto riguardano l'affidabilità dei contenuti generati dall'LLM e l'interfacciamento con i dati aziendali. Di seguito vengono analizzati i problemi specifici e le relative soluzioni adottate.

1. Recupero deterministico dei dati per l'LLM

Descrizione: gli LLM eccellono nella produzione di testo fluente, ma se incaricati anche del recupero o del calcolo dei dati tendono a produrre *allucinazioni*, ovvero affermazioni plausibili ma non supportate dai dati reali. Gli approcci più diffusi per interrogare dati aziendali tramite LLM — il RAG e le tecniche Text-to-SQL — si sono rivelati, dallo studio preliminare, inadatti ai dati strutturati: in un contesto in cui il riepilogo supporta decisioni operative, un valore inventato non è accettabile.

Soluzione: è stato introdotto un *semantic layer* (Cube), uno strato intermedio che centralizza la definizione delle metriche e fornisce KPI «certificati», recuperati in modo completamente deterministico. All'LLM viene affidata soltanto l'esposizione in linguaggio naturale di

valori già calcolati, con istruzioni che ne vincolano il comportamento ai soli dati forniti.

2. Interfacciamento tra il semantic layer e il database aziendale

Descrizione: la piattaforma conserva i dati su MongoDB, un database NoSQL, mentre Cube opera su sorgenti relazionali. L'azienda disponeva di una soluzione temporanea, applicata a un solo cliente: il travaso dei dati in un campo `jsonb` su PostgreSQL, poi «srotolato» in una vista materializzata. Nel tentativo di rendere l'architettura più generale ho sperimentato l'interrogazione diretta del campo `jsonb` grezzo da parte di Cube, ottenendo però tempi di risposta proibitivi e configurazioni SQL molto difficili da mantenere; è stata valutata anche l'esportazione dei dati in formato Parquet, tecnicamente percorribile ma onerosa. Questa problematica ha assorbito una parte significativa del tempo di stage.

Soluzione: in accordo con il tutor aziendale, l'ottimizzazione dell'interfacciamento — questione prettamente infrastrutturale — è stata messa in pausa per non bloccare l'avanzamento del progetto: lo sviluppo è proseguito sull'ambiente PostgreSQL già pronto e configurato, mantenendo l'architettura aperta a una futura generalizzazione.

3. Gestione della multitenancy

Descrizione: i database dei vari clienti hanno schemi per lo più simili, ma presentano campi personalizzati; occorre inoltre garantire che ogni cliente possa accedere esclusivamente ai propri dati.

Soluzione: per i data model è stata utilizzata la direttiva `extends` di Cube, che permette di condividere le definizioni comuni estendendole in modo granulare per ciascun cliente; la segregazione dei dati è garantita dall'autenticazione tramite token JWT prevista nativamente da Cube. Anche le interrogazioni per il recupero dei KPI sono configurate per singolo tenant su MongoDB, senza interventi sul codice del servizio.

4. Definizione del perimetro dei KPI

Descrizione: una volta risolta la questione del recupero, restava da definire quali KPI fossero utili al riepilogo e in quale forma esporli: se sviluppare vere e proprie funzioni di analisi del dato (ad esempio il calcolo di trend), o se arricchire i KPI con dati qualitativi recuperati direttamente dal database.

Soluzione: confrontandomi con il tutor aziendale ho scelto di mantenere il sistema semplice: KPI aggregati «grezzi», affidando all'LLM la loro descrizione in linguaggio naturale guidata da istruzioni specifiche per ciascun blocco di dati. L'arricchimento con dati qualitativi, pur esplorato con risultati interessanti, è stato accantonato per rimanere strettamente sui dati aggregati.

1.2.3 Pianificazione

Il progetto è stato suddiviso in fasi, dedicate dapprima alla comprensione del dominio e alla scelta dell'architettura, e in seguito allo sviluppo vero e proprio:

1. studio preliminare dell'approccio RAG e delle tecniche di interrogazione di dati tramite LLM;
2. analisi degli approcci per i dati strutturati e scelta del semantic layer;
3. sviluppo di prototipi con Cube per validarne la fattibilità: gestione della multitenancy, autenticazione JWT, interfacciamento con i dati;
4. studio del linguaggio Go;
5. sviluppo del servizio: integrazione con Cube e con le API dell'LLM, gestione dinamica delle query, caching dei riepiloghi;
6. integrazione con la coda SQS e predisposizione dell'ambiente di sviluppo locale;
7. **[DA COMPLETARE A FINE STAGE: fase conclusiva, requisiti in rinegoziazione]**

1.2.4 Obiettivi

Il piano di lavoro concordato con l'azienda definisce i seguenti obiettivi obbligatori:

- **O01** — Ricezione di eventi asincroni dalle altre componenti della piattaforma, con caricamento delle configurazioni del cliente e delle entità interessate;
- **O02** — Valutazione di KPI specifici per entità e cliente, e interfacciamento con l'IA generativa per la produzione della sintesi testuale;
- **O03** — Indicizzazione delle entità per il recupero di documenti non strutturati (approccio RAG);
- **O04** — Ricezione di richieste utente in modalità agente, con recupero documentale e risposta.

A questi si aggiunge l'obiettivo desiderabile **D01**, l'integrazione nell'architettura e nel ciclo di rilascio della piattaforma.

Il lavoro si è concentrato dapprima sul flusso relativo ai dati strutturati (O01 e O02), portandolo a un livello di completezza e solidità vicino all'integrazione in produzione. **[DA COMPLETARE A FINE STAGE: esito di O03 e O04, requisiti in rinegoziazione]**

1.3 Il prodotto finale

Il servizio realizzato copre l'intero flusso previsto per i dati strutturati: riceve gli eventi dalla coda, recupera le interrogazioni configurate per il tenant, estrae i KPI tramite il semantic layer, genera il riepilogo con

l'LLM e lo rende persistente, con la possibilità di servire le richieste successive dalla cache. Il servizio è progettato per essere configurabile per singolo cliente senza interventi sul codice ed è corredato di un ambiente di sviluppo locale che ne riproduce fedelmente l'infrastruttura di produzione.

Per un'analisi approfondita del risultato ottenuto si rimanda al Capitolo 6.

1.4 Organizzazione del testo

Questa sezione esplicita l'organizzazione del documento, descrivendo brevemente il contenuto di ogni capitolo.

Il resto del documento è organizzato come segue: il Capitolo 2 presenta gli strumenti e le tecnologie utilizzate durante lo stage; il Capitolo 3 descrive lo studio preliminare degli approcci possibili e l'analisi dei requisiti; il Capitolo 4 illustra la progettazione e lo sviluppo del servizio; il Capitolo 5 descrive le attività di verifica e validazione; il Capitolo 6 conclude il documento con una valutazione del lavoro svolto e delle prospettive future.

Riguardo la stesura del testo sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini di uso non comune vengono definiti nel glossario, situato alla fine del documento, e alla loro prima occorrenza sono indicati con la nomenclatura *termines*;
- i termini in lingua straniera o facenti parte del gergo tecnico sono evidenziati con il carattere *corsivo*;
- i nomi di funzioni, variabili e altri elementi di codice sono scritti con carattere **monospaziato**;
- le citazioni a risorse presenti nella bibliografia sono affiancate dal rispettivo numero identificativo, es. [1].

Capitolo 2

Strumenti e tecnologie utilizzate

In questo capitolo presento le tecnologie utilizzate durante lo stage, motivandone per ciascuna il ruolo all'interno del progetto.

2.1 Go

Go è il linguaggio di programmazione utilizzato da Datasoil per lo sviluppo dei servizi backend, ed è il linguaggio in cui è stato scritto l'intero servizio oggetto dello stage. Nato in Google, Go privilegia la semplicità e la leggibilità del codice: offre una sintassi essenziale, una tipizzazione statica con tempi di compilazione rapidi, una libreria standard ricca e la compilazione in un singolo eseguibile, caratteristica che ne semplifica la distribuzione. Il linguaggio si distingue inoltre per il supporto nativo alla concorrenza, tramite le *goroutine* (thread leggeri gestiti dal runtime) e i canali di comunicazione tra esse.

Non avendo esperienza pregressa con il linguaggio, una parte dello stage è stata dedicata al suo apprendimento.

2.2 MongoDB

MongoDB è un database NoSQL orientato ai documenti: i dati sono memorizzati come documenti in formato BSON (una rappresentazione binaria di JSON) raggruppati in *collection*, senza uno schema rigido imposto a priori. È il database utilizzato da Datasoil per la persistenza dei dati, organizzati secondo il modello multi-tenant: un database logico per ciascun cliente.

Nel servizio sviluppato MongoDB svolge un duplice ruolo: contiene la configurazione delle interrogazioni per il recupero dei KPI — così che ogni tenant possa definire le proprie senza modifiche al codice — e ospita la cache dei riepiloghi generati, evitando di invocare inutilmente il modello linguistico per richieste già elaborate.

2.3 Cube

Cube è un *semantic layer* open source: uno strato intermedio che si frappone tra le sorgenti dati e le applicazioni, centralizzando la definizione delle metriche di business. Invece di distribuire la logica di calcolo dei KPI tra le varie applicazioni — con il rischio di definizioni divergenti — le metriche vengono definite una sola volta nei *data model* di Cube, e le applicazioni le interrogano tramite API ottenendo valori consistenti e, in questo senso, «certificati».

Come approfondito nel Capitolo 3, l'adozione di un semantic layer è la soluzione scelta per garantire che i dati forniti al modello linguistico siano recuperati in modo deterministico. Cube è stato preferito allo sviluppo di un backend dedicato perché offre già pronte funzionalità essenziali quali il caching delle interrogazioni, le API di accesso e un sistema di sicurezza multi-tenant basato su token JWT, sfruttato nel progetto per garantire la segregazione dei dati tra i clienti.

2.4 Modelli linguistici e API OpenAI

I modelli linguistici di grandi dimensioni (Large Language Model, LLM) sono modelli di apprendimento automatico addestrati su grandi quantità di testo, capaci di comprendere e produrre linguaggio naturale. Nel progetto l'LLM è impiegato con un ruolo deliberatamente circoscritto: non gli è richiesto di calcolare o dedurre alcun dato, ma soltanto di esporre in un testo scorrevole i KPI che gli vengono forniti, seguendo istruzioni che ne vincolano il comportamento ai soli dati presenti.

L'interazione avviene tramite le API di OpenAI, utilizzando l'SDK ufficiale per Go. Ogni generazione è una chiamata indipendente, composta da un messaggio di sistema — che definisce regole e vincoli espositivi — e da un messaggio utente contenente i dati, serializzati in JSON, corredati dalle istruzioni di lettura specifiche di ciascun blocco.

2.5 Amazon SQS ed ElasticMQ

Amazon Simple Queue Service (SQS) è il servizio di code gestite di AWS, utilizzato in Datasoil per la comunicazione asincrona tra le componenti della piattaforma. Una coda disaccoppia chi produce messaggi da chi li consuma: il produttore pubblica e prosegue, il consumatore elabora al proprio ritmo, e un messaggio la cui elaborazione fallisce torna disponibile per un nuovo tentativo.

Il servizio sviluppato è un consumatore SQS, e per il consumo della coda si appoggia a un package sviluppato internamente dall'azienda, che si

fa carico dell'intero dialogo con SQS: la ricezione dei messaggi tramite *long polling*, la loro elaborazione in parallelo da parte di un numero configurabile di *worker*, la cancellazione dalla coda dei soli messaggi elaborati con successo — così che quelli falliti vengano automaticamente riproposti — e l'arresto controllato del consumatore. Al package viene fornita unicamente la funzione di elaborazione del singolo messaggio, nella quale risiede la logica specifica del servizio.

Per lo sviluppo e i test in locale è stato utilizzato ElasticMQ, un server che espone un'API compatibile con SQS: il codice del servizio rimane identico a quello di produzione, cambia soltanto l'indirizzo della coda nella configurazione.

2.6 Docker

Docker è la piattaforma di containerizzazione utilizzata per riprodurre in locale l'infrastruttura necessaria allo sviluppo: l'istanza di Cube, il database MongoDB e la coda ElasticMQ vengono eseguiti come container, garantendo un ambiente di lavoro replicabile e vicino a quello di produzione.

2.7 Librerie di supporto

Oltre alle tecnologie principali, il servizio si appoggia ad alcune librerie consolidate dell'ecosistema Go: **Viper** per la gestione della configurazione da file e variabili d'ambiente, **Resty** per le chiamate HTTP verso le API di Cube e la libreria ufficiale **mongo-driver** per l'accesso a MongoDB.

Capitolo 3

Analisi dei requisiti

In questo capitolo descrivo lo studio preliminare degli approcci possibili, i casi d'uso e la lista dei requisiti divisi per tipologia e necessità.

3.1 Studio preliminare degli approcci

3.2 Casi d'uso

3.3 Classificazione dei requisiti

3.4 Tracciamento dei requisiti

Capitolo 4

Progettazione e sviluppo

In questo capitolo descrivo l'architettura del servizio sviluppato e le principali scelte di progettazione e implementazione.

4.1 Architettura del servizio

4.2 Estrazione dei KPI

4.3 Generazione del riepilogo tramite LLM

4.4 Caching e persistenza

4.5 Integrazione con la coda

Capitolo 5

Verifica e validazione

In questo capitolo descrivo le attività di verifica e validazione svolte sul servizio.

5.1 Approccio alla verifica

5.2 Test del flusso completo

5.3 Validazione con il tutor aziendale

Capitolo 6

Conclusioni

In questo capitolo trago le conclusioni sul progetto, valutando il raggiungimento degli obiettivi e le prospettive future.

6.1 Raggiungimento degli obiettivi

6.2 Sviluppi futuri

6.3 Conoscenze acquisite

6.4 Valutazione personale

Glossario

Bibliografia